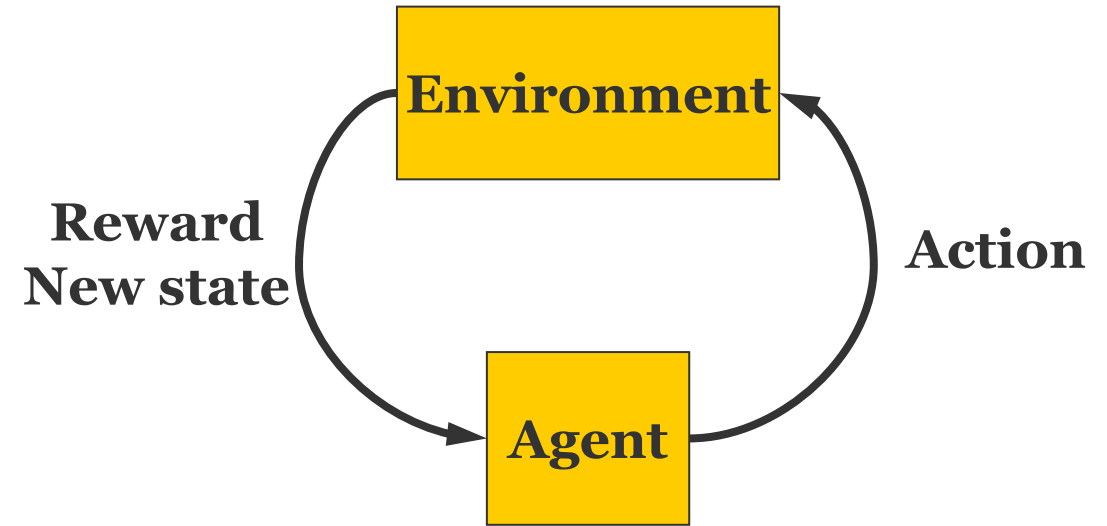


Lecture 20: Markov Processes

Instructor: Sergei V. Kalinin

(A Peek at) Reinforcement Learning

- **Supervised learning**
 - Classification
 - Regression
- **Unsupervised learning**
 - Clustering
 - Dimensionality reduction
- **Reinforcement learning**
 - more general than supervised/unsupervised learning
 - learn from interaction with environment to achieve a goal



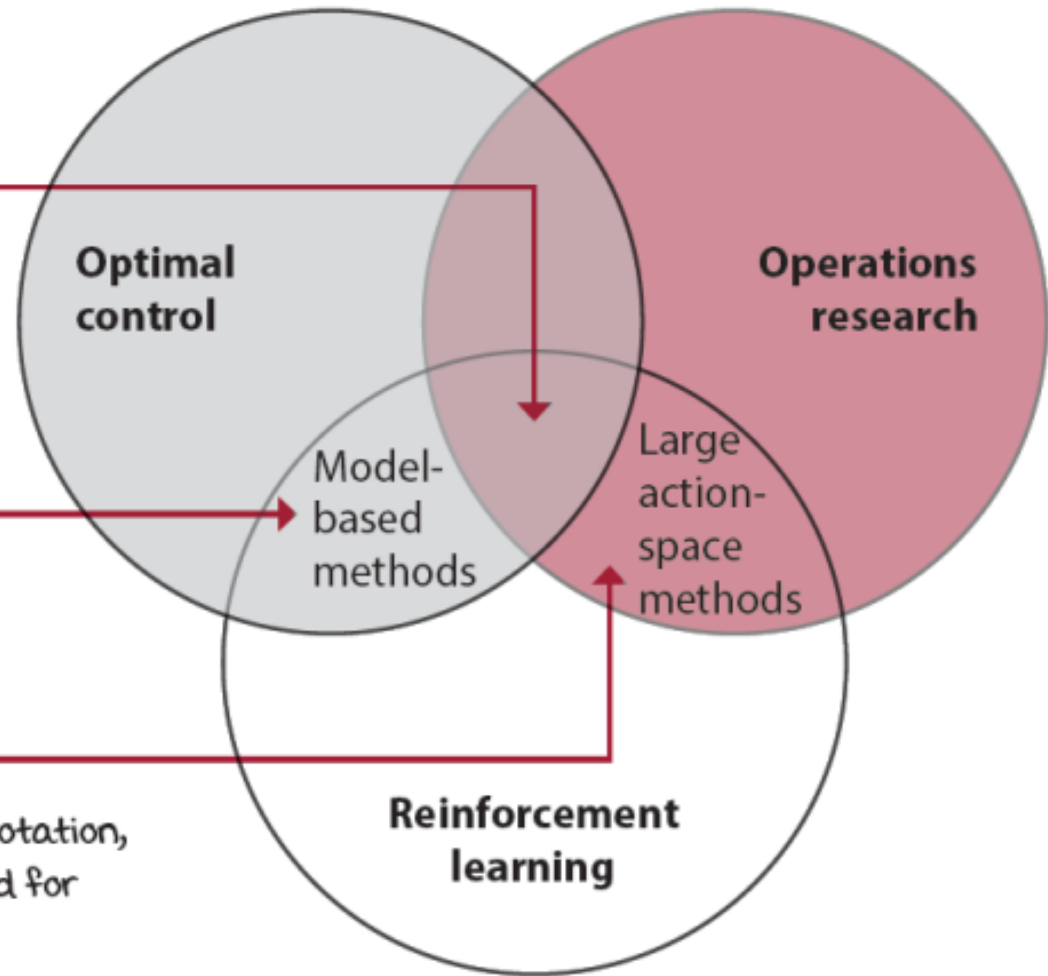
Where is reinforcement learning in ML world?

(1) All of these fields (and many more) study complex sequential decision-making under uncertainty.

(2) As a result, there's a synergy between these fields. For instance, reinforcement learning and optimal control both contribute to the research of model-based methods.

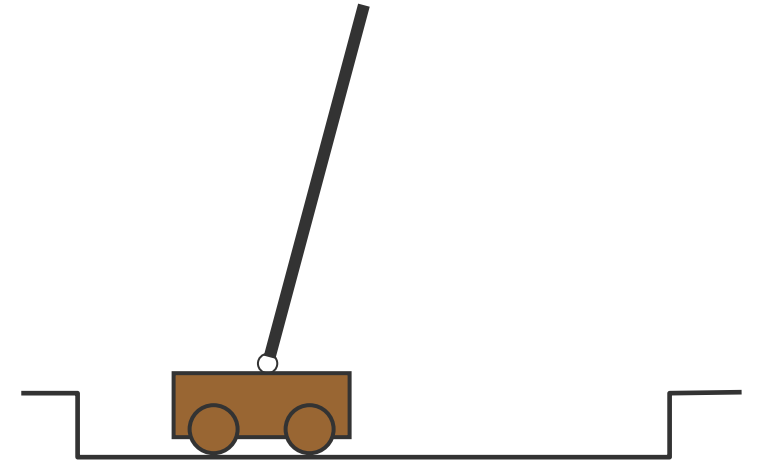
(3) Or reinforcement learning and operations research both contribute to the study of problems with large action spaces.

(4) The downside is an inconsistency in notation, definitions, and so on, that makes it hard for newcomers to find their way around.



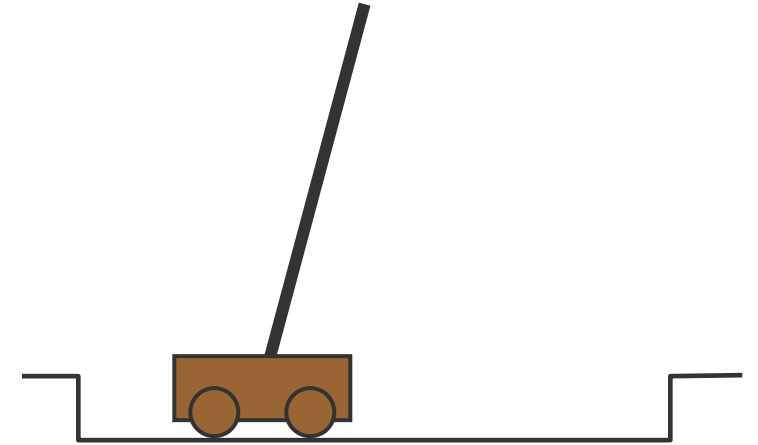
Examples

- Frozen lake
 - Pole-balancing
 - TD-Gammon [Gerry Tesauro]
 - Helicopter [Andrew Ng]
-
- No teacher who would say “good” or “bad”
 - is reward “10” good or bad?
 - rewards could be delayed
 - Similar to control theory
 - more general, fewer constraints
 - Explore the environment and learn from experience
 - not just blind search, try to be smart about it



State Representation

- Pole-balancing
 - move car left/right to keep the pole balanced
- State representation
 - position and velocity of car
 - angle and angular velocity of pole
- What about *Markov property*?
 - would need more info
 - noise in sensors, temperature, bending of pole
- Solution
 - coarse discretization of 4 state variables
 - left, center, right
 - totally non-Markov, but still works



Designing Rewards

- Robot in a maze
 - episodic task, not discounted, +1 when out, 0 for each step
- Chess
 - GOOD: +1 for winning, -1 losing
 - BAD: +0.25 for taking opponent's pieces: high reward even when lose
- Rewards
 - rewards indicate what we want to accomplish
 - NOT how we want to accomplish it



- Shaping
 - positive reward often very “far away”
 - rewards for achieving subgoals (domain knowledge)
 - also: adjust initial policy or initial value function

Robot in the room

			+1
			-1
START			

actions: UP, DOWN, LEFT, RIGHT

UP

80%

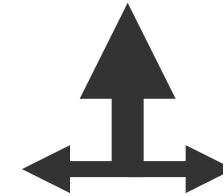
10%

10%

move UP

move LEFT

move RIGHT



- Reward +1 at [4,3], -1 at [4,2]
- Reward -0.04 for each step
- What's the strategy to achieve max reward?
- What if the actions were deterministic?

Is this a solution?

→	→	→	+1
↑			-1
↑			

- Only if actions deterministic
- What if actions are stochastic?

Solution: policy mapping from each state to an action

Optimal policy

→	→	→	+1
↑		↑	-1
↑	←	←	←

What if the reward for each step is -2?

→	→	→	+1
↑		→	-1
→	→	→	↑

Reward for each step is -0.1

→	→	→	+1
↑		↑	-1
↑	→	↑	←

Reward for each step is -0.04

→	→	→	+1
↑		↑	-1
↑	←	←	←

Reward for each step is -0.01

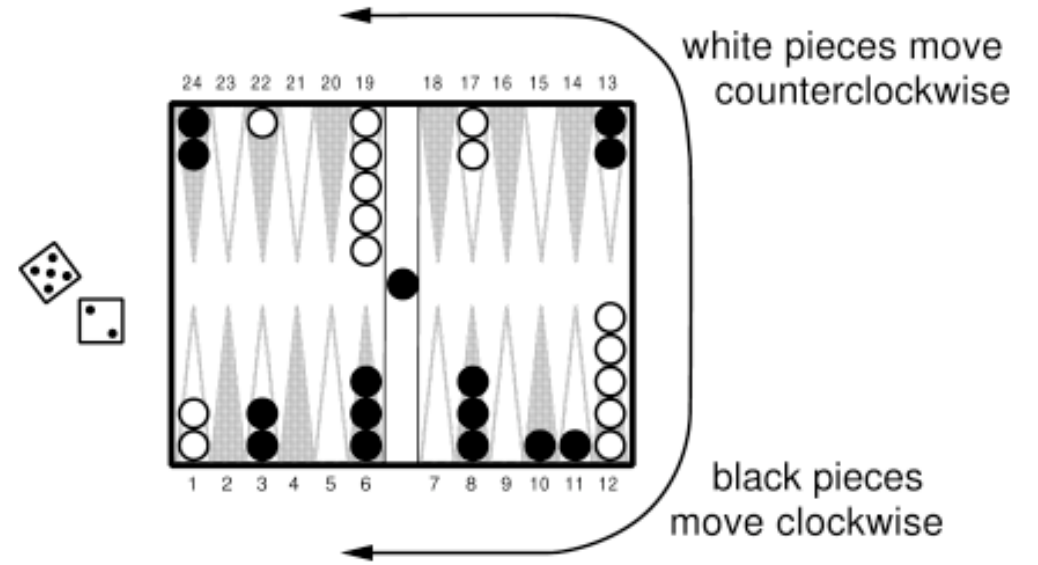
→	→	→	+1
↑		←	-1
↑	←	←	↓

Reward for each step is +0.01

↓	←	←	+1
↓		←	-1
←	←	←	↓

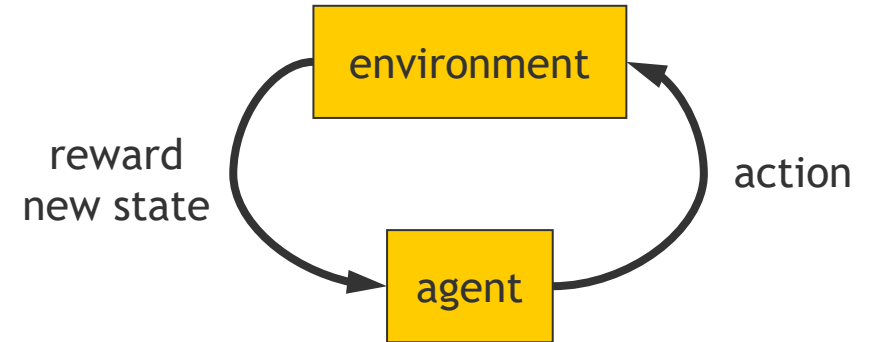
Backgammon

- Rules
 - 30 pieces, 24 locations
 - roll 2, 5: move 2, 5
 - hitting, blocking
 - branching factor: 400
- Implementation
 - use $TD(\lambda)$ and neural nets
 - 4 binary features for each position on board (# white pieces)
 - no BG expert knowledge
- Results
 - TD-Gammon 0.0: trained against itself (300,000 games)
 - as good as best previous BG computer program (also by Tesauro)
 - lot of expert input, hand-crafted features
 - TD-Gammon 1.0: add special features
 - TD-Gammon 2 and 3 (2-ply and 3-ply search)
 - 1.5M games, beat human champion

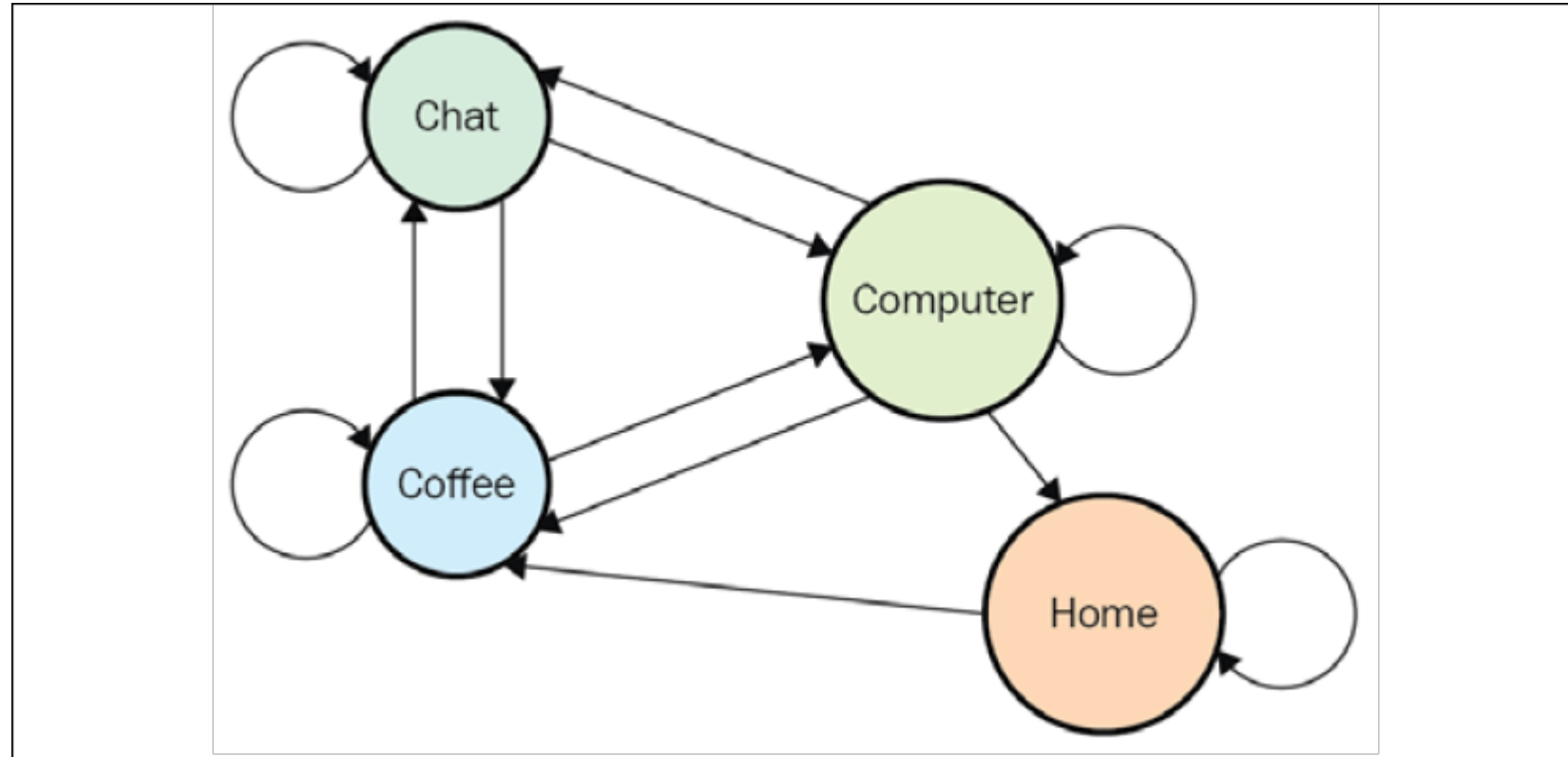


Markov Decision Processes (MDP)

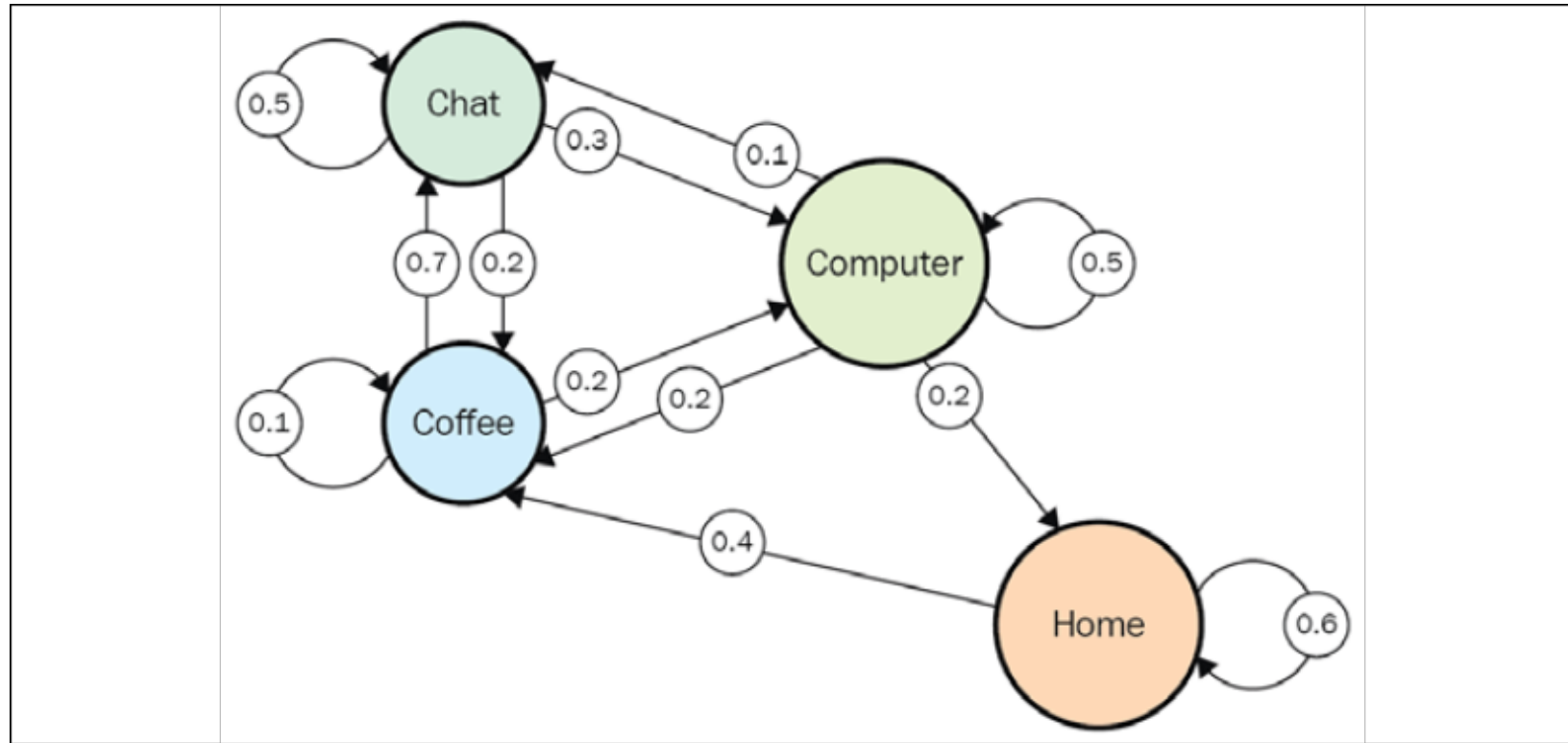
- Set of states S , set of actions A , initial state S_0
- Transition model $P(s,a,s')$
 - $P([1,1], \text{up}, [1,2]) = 0.8$
- Reward function $r(s)$
 - $r([4,3]) = +1$
- **Goal:** maximize cumulative reward in the long run
- **Policy:** mapping from S to A
 - $\pi(s)$ or $\pi(s,a)$ (deterministic vs. stochastic)
- **Reinforcement learning:**
 - transitions and rewards usually not available
 - how to change the policy based on experience
 - how to explore the environment



Let's define states



And transition probabilities



Finite Markov chain

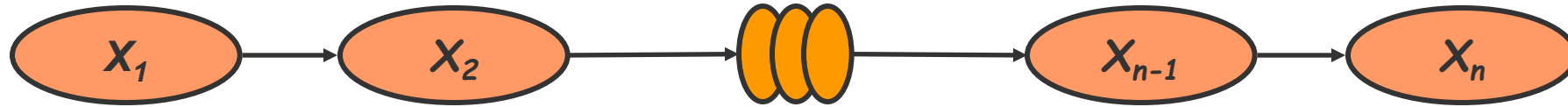
An *integer time stochastic process*, consisting of a **domain D** of **$m > 1$** states $\{s_1, \dots, s_m\}$ and

1. An **m** dimensional *initial distribution vector* $(p(s_1), \dots, p(s_m))$.
2. An **$m \times m$** *transition probabilities matrix* $M = (a_{s_i s_j})$

For example,

- **D** can be the letters $\{A, C, T, G\}$,
- $p(A)$ the probability of A to be the 1st letter in a sequence, and
- a_{AG} the probability that G follows A in a sequence.

Markov Chain



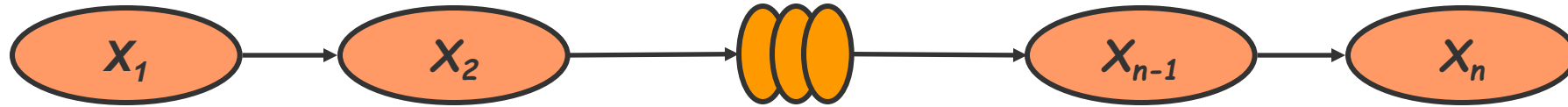
For each integer n , a Markov Chain assigns probability to sequences $(x_1 \dots x_n)$ over $\mathbf{D}$ (i.e, $x_i \in \mathbf{D}$) as follows:

$$p((x_1, x_2, \dots, x_n)) = p(X_1 = x_1) \prod_{i=2}^n p(X_i = x_i \mid X_{i-1} = x_{i-1}) = p(x_1) \prod_{i=2}^n a_{x_{i-1}x_i}$$

Similarly, $(\mathbf{X}_1, \dots, \mathbf{X}_i, \dots)$ is a sequence of probability distributions over $\mathbf{D}$.

There is a rich theory which studies the properties of these sequences.

Markov Chain



Similarly, each $\mathbf{X}_i$ is a probability distributions over $\mathbf{D}$, which is determined by the initial distribution $(p_1, \dots, p_n)$ and the transition matrix $\mathbf{M}$.

There is a rich theory which studies the properties of such “Markov sequences” $(\mathbf{X}_1, \dots, \mathbf{X}_i, \dots)$.

Matrix Representation

	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

The transition probabilities
Matrix $\mathbf{M} = (a_{st})$

$\mathbf{M}$ is a stochastic Matrix:

$$\sum_t a_{st} = 1$$

The initial ***distribution vector***
($u_1 \dots u_m$) defines the distribution of
 $\mathbf{X}_1$ ($p(\mathbf{X}_1 = s_i) = u_i$).

Then after one move, the distribution is changed to $\mathbf{X}_2 = \mathbf{X}_1 \mathbf{M}$

Matrix Representation

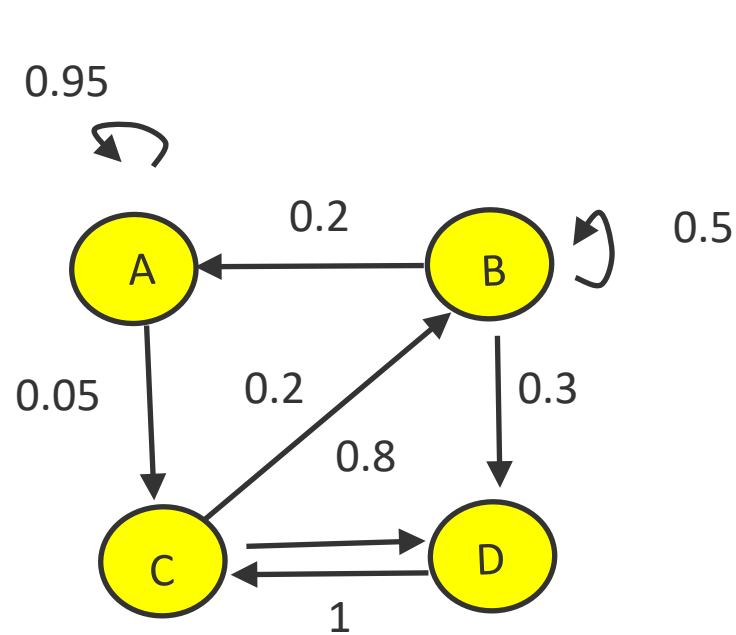
	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

Example: if $\mathbf{X}_1 = (0, 1, 0, 0)$
then $\mathbf{X}_2 = (0.2, 0.5, 0, 0.3)$

And if $\mathbf{X}_1 = (0, 0, 0.5, 0.5)$
then $\mathbf{X}_2 = (0, 0.1, 0.5, 0.4)$.

The i -th distribution is $\mathbf{X}_i = \mathbf{X}_1 \mathbf{M}^{i-1}$

Representation of Markov Chain as Digraph



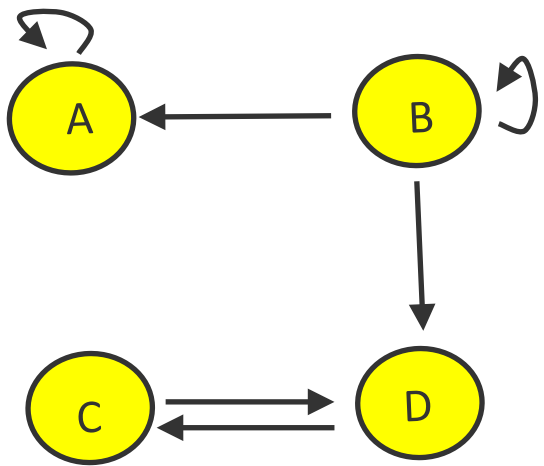
	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

Each directed edge $A \rightarrow B$ is associated with the **positive** transition probability from A to B.

Properties of Markov Chain States

States of Markov chains are classified by the digraph representation (omitting the actual probability values)

A, C and D are **recurrent** states: they are in strongly connected components which are **sinks** in the graph.

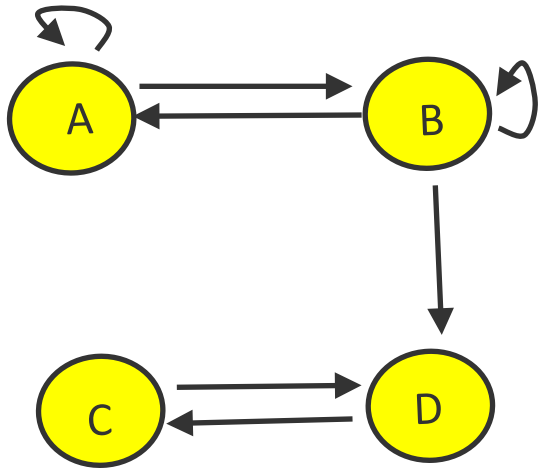


B is not recurrent – it is a **transient** state

Alternative definitions:

A state **s** is **recurrent** if it can be reached from any state reachable from **s**; otherwise it is **transient**.

Another example: transient and recurrent states

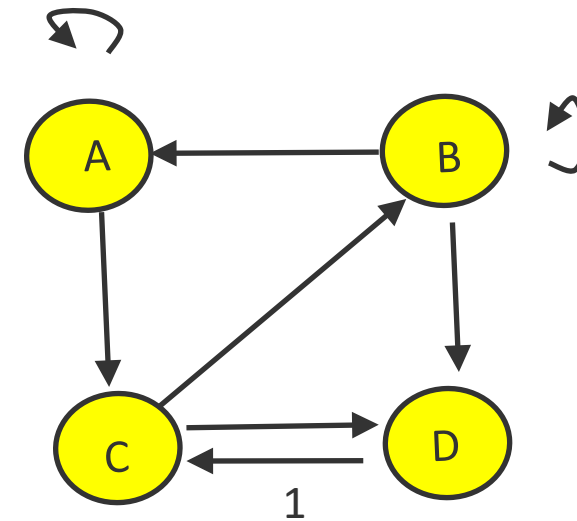
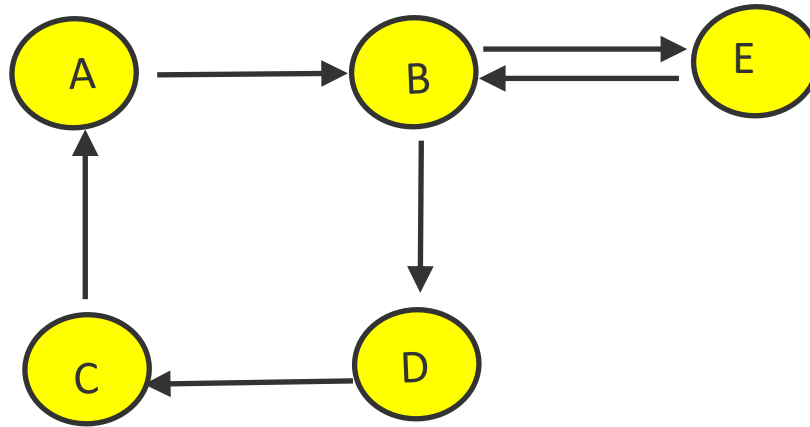


A and **B** are *transient* states, **C** and **D** are *recurrent* states.

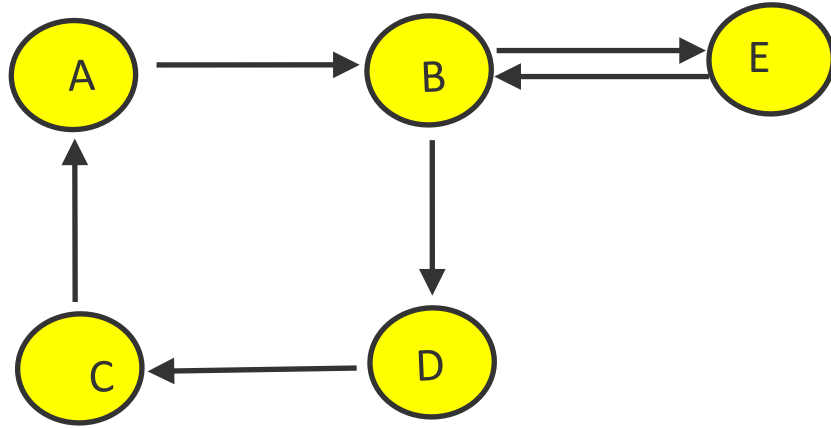
Once the process moves from **B** to **D**, it will never come back.

Irreducible Markov Chain

A Markov Chain is ***irreducible*** if the corresponding graph is strongly connected (and thus all its states are recurrent).



Periodic States



A state s has a period k if k is the **GCD** of the lengths of all the cycles that pass via s .

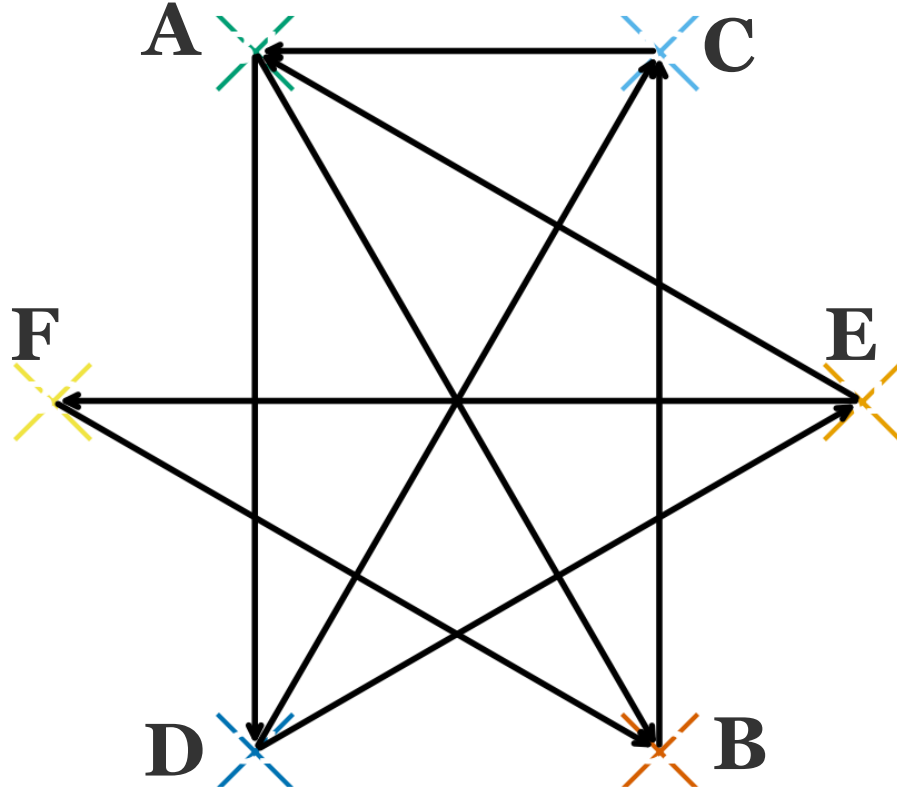
In the shown graph the period of A is 2.

All the states in the same strongly connected component have the same period

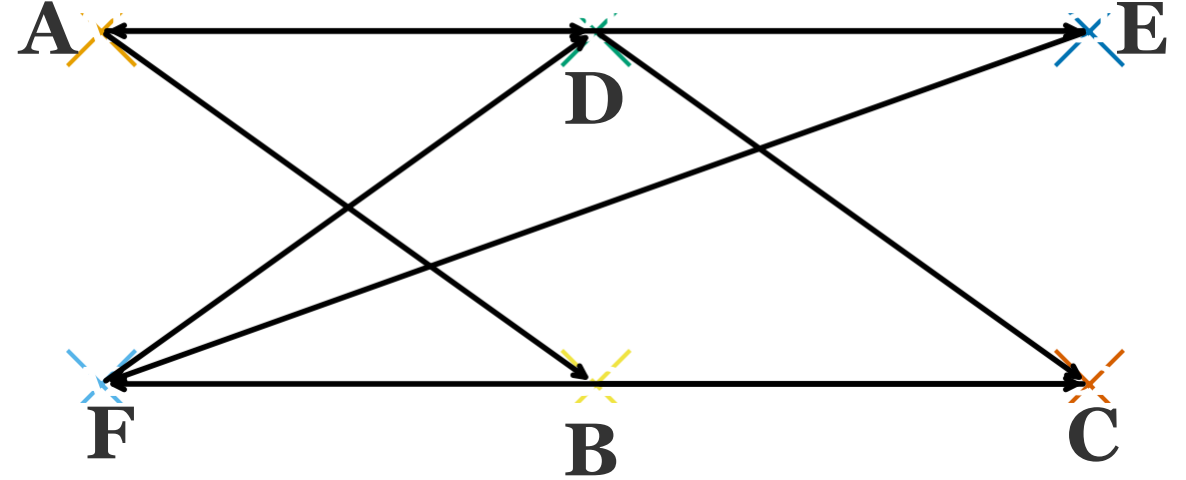
A Markov Chain is **periodic** if all the states in it have a period $k > 1$. It is **aperiodic** otherwise.

Periodic Chain: $k = 3$

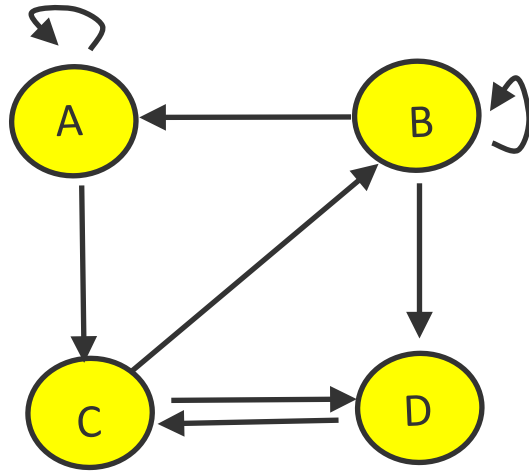
Period-3 Chain (generic layout)



Period-3 Chain (tripartite layout: C0 | C1 | C2)



Ergodic Markov Chain



A Markov chain is ***ergodic*** if :

- 1. the corresponding graph is strongly connected.***
- 2. It is not periodic***

Ergodic Markov Chains are important since they guarantee the corresponding Markovian process converges to a unique distribution, in which all states have strictly positive probability.

Stationary Distribution for Markov Chain

Let $\mathbf{M}$ be a Markov Chain of m states, and let $\mathbf{V} = (v_1, \dots, v_m)$ be a probability distribution over the m states

$\mathbf{V} = (v_1, \dots, v_m)$ is **stationary distribution** for $\mathbf{M}$ if $\mathbf{VM} = \mathbf{V}$.

(i.e., if one step of the process does not change the distribution).

$\mathbf{V}$ is a stationary distribution



$\mathbf{V}$ is a left (row) Eigenvector of $\mathbf{M}$ with Eigenvalue 1.

Good Markov Chain

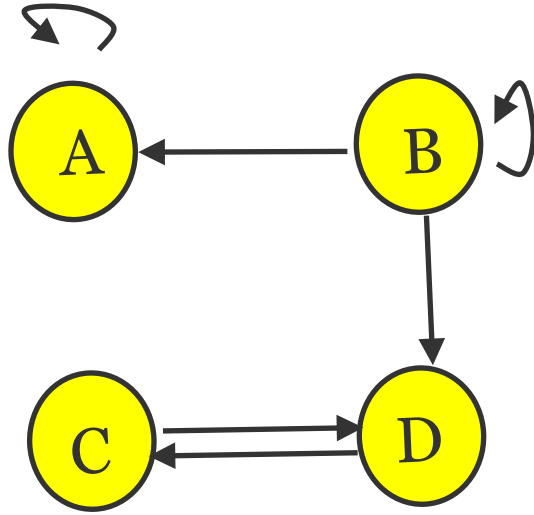
A Markov Chain is *good* if the distributions X_i , as $I \rightarrow \infty$:

- (1) converge to a unique distribution, independent of the initial distribution.
- (2) In that unique distribution, each state has a positive probability.

The Fundamental Theorem of Finite Markov Chains:

A Markov Chain is good $\Leftrightarrow$ the corresponding graph is ergodic.

Bad Markov Chains



Consider two initial distributions:

a) $p(\mathbf{X}_1=A)=1$ ($p(\mathbf{X}_1 = \mathbf{x})=0$ if $\mathbf{x}\neq A$).

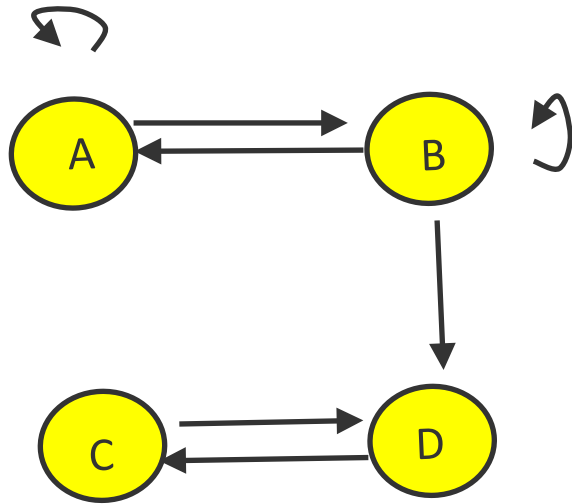
b) $p(\mathbf{X}_1= C) = 1$

In case a), the sequence will stay at A forever.

In case b), it will stay in $\{C,D\}$ for ever.

If G has two states which are unreachable from each other, then $\{\mathbf{X}_i\}$ cannot converge to a distribution which is independent on the initial distribution.

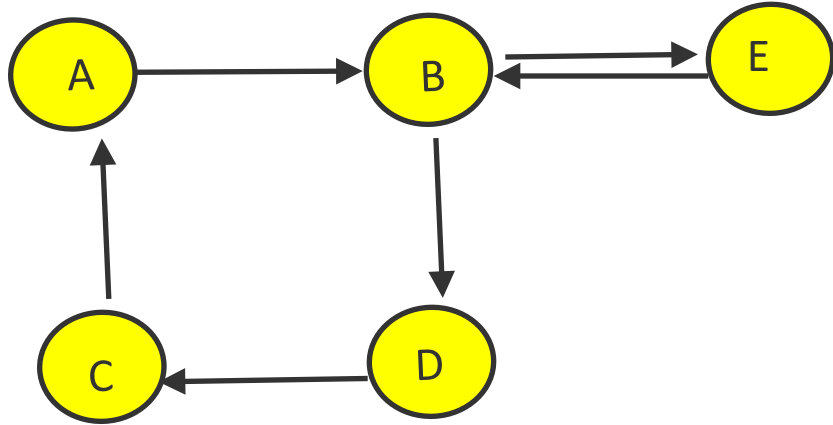
Transient state



Once the process moves from **B** to **D**, it will never come back.

For each initial distribution, with probability 1 a transient state will be visited only a finite number of times.

Periodic Chain



In a periodic Markov Chain (of period $k > 1$) there are initial distributions under which the states are visited in a periodic manner.

Under such initial distributions $\mathbf{X}_i$ does not converge as $i \rightarrow \infty$.

Corollary: A good Markov Chain is not periodic

Hidden Markov Model

- A Hidden Markov model is a statistical model in which the system being modelled is assumed to be Markov process with unobserved hidden states.
- In regular Markov models the state is clearly visible to others in which the state transition probabilities are the parameters only whereas in HMM the state is not visible but the output is visible.

Hidden Markov Model

- It consists of set of states : $S_1, S_2, S_3, \dots, S_n$.
- Process moves from one state to another state generating a sequence of states $S_{i1}, S_{i2}, \dots, S_{ik}$...
- Markov chain property: probability of each subsequent state depends only on what was the previous state

$$P(S_{ik} | S_{k1}, S_{k2}, \dots, S_{k-1}) = P(S_{ik} | S_{k-1})$$

- States are not visible, but each state randomly generates one of M observations (or visible states)

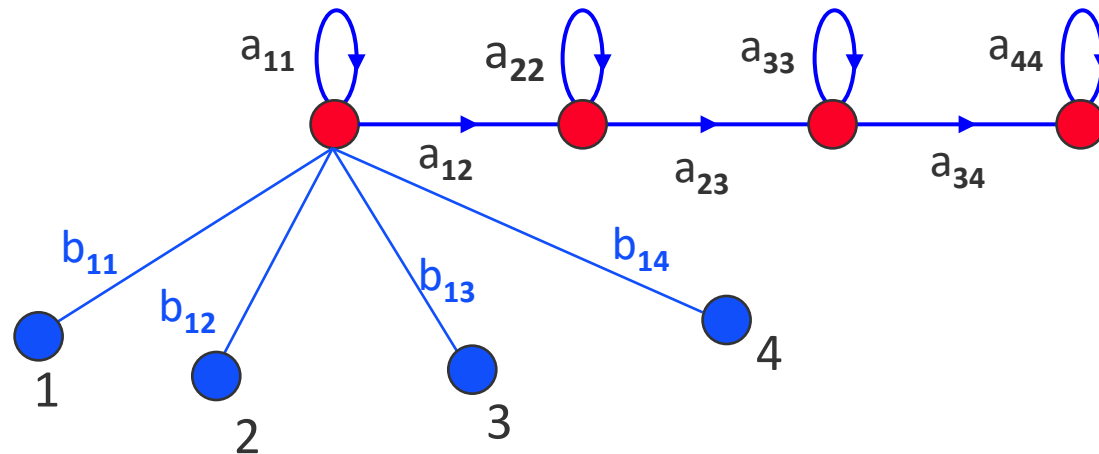
$$V = \{v_1, v_2, v_3, \dots, v_k, \dots\}$$

Hidden Markov Model

To define hidden Markov model, the following probabilities have to be specified:

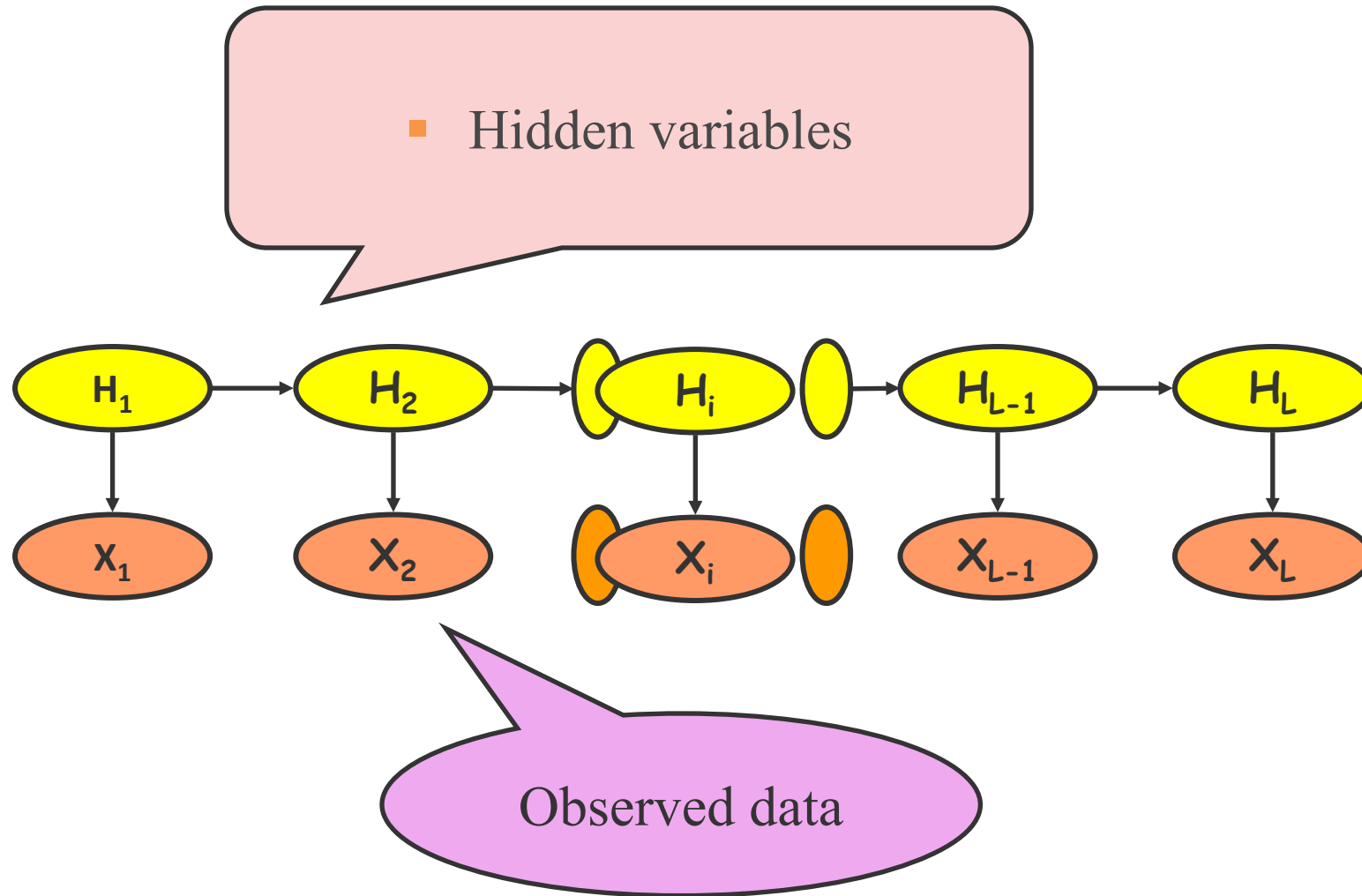
- matrix of transition probabilities $A=(a_{ij})$, $a_{ij}= P(s_i | s_j)$,
- matrix of observation probabilities $B=(b_i(v_m))$, $b_i(v_m) = P(v_m | s_i)$, and
- a vector of initial probabilities $p=(\pi_i)$, $p_i = P(s_i)$

Model is represented by $M=(A, B, p)$.



- a_{ij} are state transition probabilities.
- b_{ik} are observation (output) probabilities.
- $b_{11} + b_{12} + b_{13} + b_{14} = 1$,
- $b_{21} + b_{22} + b_{23} + b_{24} = 1$.

Hidden Markov Model



Main Problems

- **Evaluation problem:** Given the HMM $M = \{ A, B, p \}$ and observation sequence $O = o_1, o_2, \dots, o_k$, calculate the probability that model m has generated sequence O .
- **Decoding problem :** Given the HMM $M = \{ A, B, p \}$ and observation sequence $O = o_1, o_2, \dots, o_k$, Calculate the most likely sequence of hidden states S_i that generated sequence O .
- **Learning Problem :** Given some training observation sequences $O = o_1, o_2, \dots, o_k$, and general structure of HMM(visible and hidden states), determine HMM parameters that best fit the training data.